

AI Reconciliation Platform — Final System Blueprint (FastAPI + React Architecture)

OVERARCHING ARCHITECTURE

This system is a forward-only, API-driven reconciliation engine.

Design Pillars:

- API-first architecture
- Backend-owned state machine
- Forward-only state transitions
- Immutable phase-level snapshots
- Deterministic authority layer
- Probabilistic additive layer
- AI advisory layer (non-authoritative)
- Human governance with preserved overrides
- No backward navigation
- No recomputation during review phases
- Snapshot created before every state transition

Tooling Alignment:

- Backend Development: VS Code + Claude Code
 - Frontend Development: Cursor (React + Tailwind)
 - Runtime: FastAPI (Uvicorn)
 - Testing: Pytest
 - Deployment: Azure App Service (API) + Azure Static Web Apps (Frontend)
 - Storage (Phase 2+): Azure Blob Storage + Optional DB
-

GLOBAL STATE MACHINE

Valid States:

- initialized
- files_loaded
- profiled
- preprocessed
- deterministic_complete
- deterministic_review_complete
- probabilistic_complete
- probabilistic_review_complete

- ai_suggested
- ai_review_complete
- final_consolidated
- finalized

Rules:

- Strict forward-only transitions
 - State validated on every API call
 - Snapshot written immediately before advancing state
 - No state mutation outside transition functions
-

RUNTIME CONTAINER STRUCTURE (Backend-Owned)

Runtime Object:

```
runtime = {  
  "session_id": str,  
  "current_state": str,  
  "raw_data": {  
    "chart_of_accounts": DataFrame,  
    "gl": DataFrame,  
    "subledger": DataFrame  
  },  
  "clean_data": {},  
  "vendor_normalization_map": [],  
  "profiling": {},  
  "matching": {  
    "deterministic": [],  
    "probabilistic": [],  
    "ai_suggested": [],  
    "final": [],  
    "rejected": []  
  },  
  "residual_pool": {},  
  "snapshots": {},  
  "config": {  
    "weights": {},  
    "threshold": None,  
    "ai_model": None,  
    "vendor_nlp_threshold": 0.90  
  }  
}
```

Storage Strategy:

- MVP: In-memory dict keyed by session_id
 - Phase 2: Redis or DB-backed persistence
-

PHASE-BY-PHASE SYSTEM BLUEPRINT

PHASE 0 — INITIALIZATION

State: initialized

Tool:

- Backend (FastAPI)

Objective:

- Create runtime container
- Issue session_id

Specifics:

- POST /reconciliation/start
- Initialize runtime object
- No data loaded
- No snapshots yet

Outputs:

- session_id
 - current_state
-

PHASE 1 — DATA INTAKE

State: files_loaded

Tool:

- Frontend: React file upload UI
- Backend: FastAPI schema validation service

Objective:

- Upload and validate required datasets

Required Files:

- Chart_of_Accounts.csv

- GL.csv
- Subledger.csv

Schema Requirements:

- Required columns enforced
- Data types enforced
- No silent coercion
- Row-level validation errors surfaced

Snapshot Contents:

- Deep copy of validated raw data
- Row counts
- Schema hash

Outputs:

- Row counts
 - Schema validation summary
-

PHASE 2 — DATA PROFILING

State: profiled

Tool:

- Backend profiling service
- Optional AI narrative service

Objective:

- Generate deterministic profiling metrics
- Provide risk visibility

Deterministic Metrics:

- Row counts
- Null percentages
- Duplicate counts
- Date ranges
- Amount distributions
- Entity distributions

AI Narrative (Advisory Only):

- Anomaly summary
- Concentration risk flags
- Vendor exposure insights

Snapshot Includes:

- Metrics
- Narrative

Outputs:

- Profiling metrics object
 - Narrative summary
-

PHASE 3 — PREPROCESSING (Vendor Normalization Pipeline)

State: preprocessed

Tool:

- Backend transformation service
- Deterministic normalization utilities
- NLP similarity engine (threshold-driven)
- AI vendor resolution service (residual only)

Objective:

- Produce authoritative Vendor_Normalized column
 - Create auditable vendor normalization mapping table
 - Standardize vendor identities prior to matching
 - Ensure downstream deterministic/probabilistic matching uses normalized vendor field
-

Vendor Normalization Pipeline (Three-Tiered)

Tier 1 — Deterministic Preprocessing

- Lowercasing
- Punctuation removal
- Whitespace normalization
- Corporate suffix stripping (LLC, INC, LTD, CORP, CO)
- Stopword removal (THE, COMPANY, SERVICES where appropriate)
- Abbreviation expansion table
- Explicit alias mapping table (config-driven)

All direct matches from Tier 1 are tagged:

- match_source = "preprocessing"
-

Tier 2 — NLP Similarity Matching

Purpose:

Resolve near-identical vendor strings not captured by deterministic rules.

Engine:

- Token normalization
- Token sorting
- RapidFuzz / cosine similarity (deterministic NLP)

Threshold:

- `runtime["config"]["vendor_nlp_threshold"] = 0.90` (default)

Rules:

- Similarity $\geq 0.90 \rightarrow$ auto-normalize
- Similarity $< 0.90 \rightarrow$ move to AI residual bucket
- Grouping performed only when similarity threshold satisfied

All NLP matches are tagged:

- `match_source = "nlp"`
 - `similarity_score` stored in mapping table
-

Tier 3 — AI Residual Vendor Resolution

Purpose:

Resolve vendors not matched in Tier 1 or Tier 2.

Inputs:

- Residual vendor strings
- Context fields (entity, frequency, amount totals)

AI Role:

- Suggest canonical vendor label
- Suggest grouping candidates
- Provide confidence score

Governance:

- AI suggestions must be confirmed (auto-accept allowed if confidence $>$ configurable threshold in future)
- No mutation without logging

All AI matches are tagged:

- `match_source = "ai"`
 - `ai_confidence_score` logged
 - model name + prompt version stored in snapshot
-

Vendor_Normalized Mapping Table (Authoritative Output of Phase 3)

Structure:

```
vendor_normalization_map = [
{
"original_vendor": str,
"vendor_normalized": str,
"match_source": "preprocessing" | "nlp" | "ai",
"similarity_score": float | null,
"ai_confidence_score": float | null,
"created_at": timestamp
}
]
```

Rules:

- One row per distinct original vendor string
 - Immutable once Phase 3 snapshot created
 - Versioned via snapshot hash
-

Vendor_Normalized Column Propagation

After mapping table creation:

- Add "Vendor_Normalized" column to:
 - GL table
 - Subledger table

Mapping Logic:

- Left join on original vendor field
- If no mapping found → Vendor_Normalized = original vendor
- All downstream deterministic and probabilistic matching uses Vendor_Normalized
- Original vendor column preserved for audit

This ensures:

- AWS and Amazon Web Services normalize consistently
 - Deterministic layer benefits from canonical identity
 - NLP and AI augment preprocessing without corrupting raw data
-

Governance:

- Explicit POST /preprocess required
- Full mapping table generated before state transition
- Transformation log records:
 - Alias table version
 - NLP threshold used
 - AI model used (if invoked)
 - Count of preprocessing / NLP / AI matches

Snapshot Includes:

- Clean datasets
- Full vendor_normalization_map
- Vendor counts before vs after normalization
- Config metadata (thresholds + AI model)

Outputs:

- Cleaned dataset metadata
 - Canonical vendor summary metrics
 - Vendor normalization mapping table
-

PHASE 4 — DETERMINISTIC MATCHING

State: deterministic_complete

Tool:

- Backend deterministic engine

Objective:

- Authoritative match layer

Scenario Hierarchy (Ordered Execution):

1. Strict 1:1 (Exact vendor, amount, date, entity)
2. Vendor + Amount + Date tolerance (60 days)
3. Amount + Entity + Date tolerance
4. Deterministic grouping (Exact sum equality)

Rules:

- Once matched, removed from pool
- Confidence score assigned
- No overrides allowed in v1

Match Object Schema:

- match_id
- record_ids_A
- record_ids_B
- scenario_id
- scenario_description
- confidence_score
- grouping_type
- user_status
- override_flag

Snapshot Includes:

- Full deterministic matches
- Residual unmatched pool

Outputs:

- Deterministic match list
 - Residual count
-

PHASE 4A — DETERMINISTIC REVIEW

State: deterministic_review_complete

Tool:

- Backend state transition only

Objective:

- Confirm deterministic layer

Rules:

- Confirm-only review
- No rejection in v1
- No recomputation

Snapshot Created Before Advancement

PHASE 5 — PROBABILISTIC MATCHING

State: probabilistic_complete

Tool:

- Backend similarity engine

Objective:

- Match residual pool via similarity scoring

Blocking Strategy:

- Same entity
- Date within 60 days
- Amount magnitude proximity band

Similarity Formula:

final_similarity =
0.40 * vendor_similarity +
0.35 * amount_similarity +
0.15 * date_similarity +
0.10 * entity_similarity

Dual Tolerance for Amount:

- Percent tolerance
- Absolute tolerance

Grouping Rules:

- Max grouping size: 5
- Greedy bounded search
- Highest similarity first
- Remove matched from pool

Threshold:

- User-provided
- Stored in config

Snapshot Includes:

- Accepted matches
- Residual pool
- Threshold and weights

Outputs:

- Probabilistic match list
 - Residual count
-

PHASE 5A — PROBABILISTIC REVIEW

State: probabilistic_review_complete

Tool:

- Backend decision processor

Objective:

- Human governance over similarity layer

User Actions:

- Accept
- Reject

Rules:

- Rejected matches moved to rejected dataset
- Records returned to residual pool
- No recomputation triggered

Snapshot Includes:

- Match statuses
- Rejected collection

Outputs:

- Accepted count
 - Rejected count
-

PHASE 6 — AI SUGGESTED MATCHES

State: ai_suggested

Tool:

- Backend AI service (OpenAI API via ai_layer.py)

Objective:

- Provide advisory-only suggestions

Inputs:

- Residual unmatched
- Near-threshold probabilistic candidates

AI Match Object Schema:

- match_id
- record_ids_A
- record_ids_B
- ai_confidence_score
- materiality
- supporting_features
- reasoning_narrative
- grouping_type
- user_status (pending)
- override_flag

Ranking:

1. Materiality (descending)
2. AI confidence (descending)

AI Governance Rules:

- No automatic acceptance
- No dataset mutation
- Model + prompt version logged in snapshot

Snapshot Includes:

- AI matches

- Model metadata

Outputs:

- AI suggestion list
-

PHASE 6A — AI REVIEW

State: ai_review_complete

Tool:

- Backend decision processor

Objective:

- Human acceptance or rejection of AI matches

Rules:

- Accepted AI matches move to final layer
- Rejected stored in rejected dataset
- No recomputation

Snapshot Includes:

- AI match statuses

Outputs:

- Accepted AI count
 - Rejected AI count
-

PHASE 7 — FINAL CONSOLIDATION

State: final_consolidated

Tool:

- Backend consolidation service

Objective:

- Produce final reconciliation view

Final Dataset Includes:

- Deterministic (confirmed)
- Accepted Probabilistic
- Accepted AI

Additional Outputs:

- Residual unmatched
- Rejected matches
- Override summary

Snapshot Includes:

- Final matches
- Residual
- Rejected
- Summary metrics

Outputs:

- Final match count
 - Residual count
 - Rejected count
-

PHASE 8 — FINAL OUTPUT

State: finalized

Tool:

- Backend export service

Objective:

- Generate audit-ready deliverables

Deliverables:

CSV Files:

- Final accepted matches
- Residual unmatched
- Rejected matches
- Full audit log

PDF Report:

- Executive summary
- Reconciliation statistics
- Override summary
- AI narrative insights

Storage Strategy:

- Local temp (MVP)
- Blob Storage (Phase 2+)

Outputs:

- Download URLs
-

SNAPSHOT SPECIFICATION

Each snapshot contains:

- Deep copy of state data
- Config metadata (weights, thresholds, AI model)
- Row counts
- Integrity hash
- Narrative (if applicable)
- Timestamp

Guarantees:

- Reproducible
 - Immutable
 - Auditable
-

GLOBAL GUARANTEES

- No backward navigation
 - No silent data mutation
 - No recomputation during review
 - Deterministic layer immutable
 - Overrides preserved
 - AI advisory only
 - Audit trace reproducible
 - Clear separation of frontend and backend responsibilities
-

Tooling

Layer	Tool
Backend Dev	VS Code + Claude
Frontend Dev	Cursor
Terminal/Automation	Warp

Cloud Runtime	Azure
Version Control	GitHub
Testing	Pytest

